

Loop Control Statements

Loop control statements are used when a section of code may either be executed a fixed number of times, or while some condition is true.

- While
- For

while Loop

The while loop keeps repeating an action until an associated condition returns false.

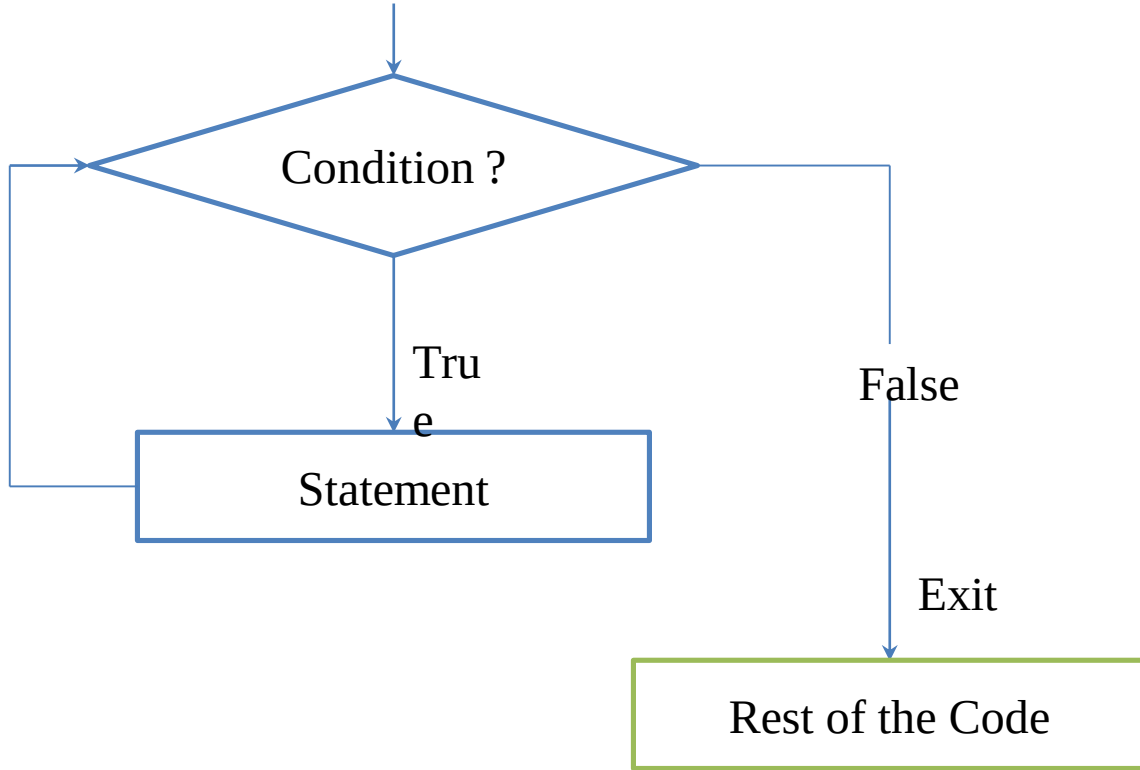
Syntax:

```
while (condition):  
    Statement  
Rest of the Code
```

Python Interpreter checks condition,
If condition is True, then

Execute statements written after
colon (:)



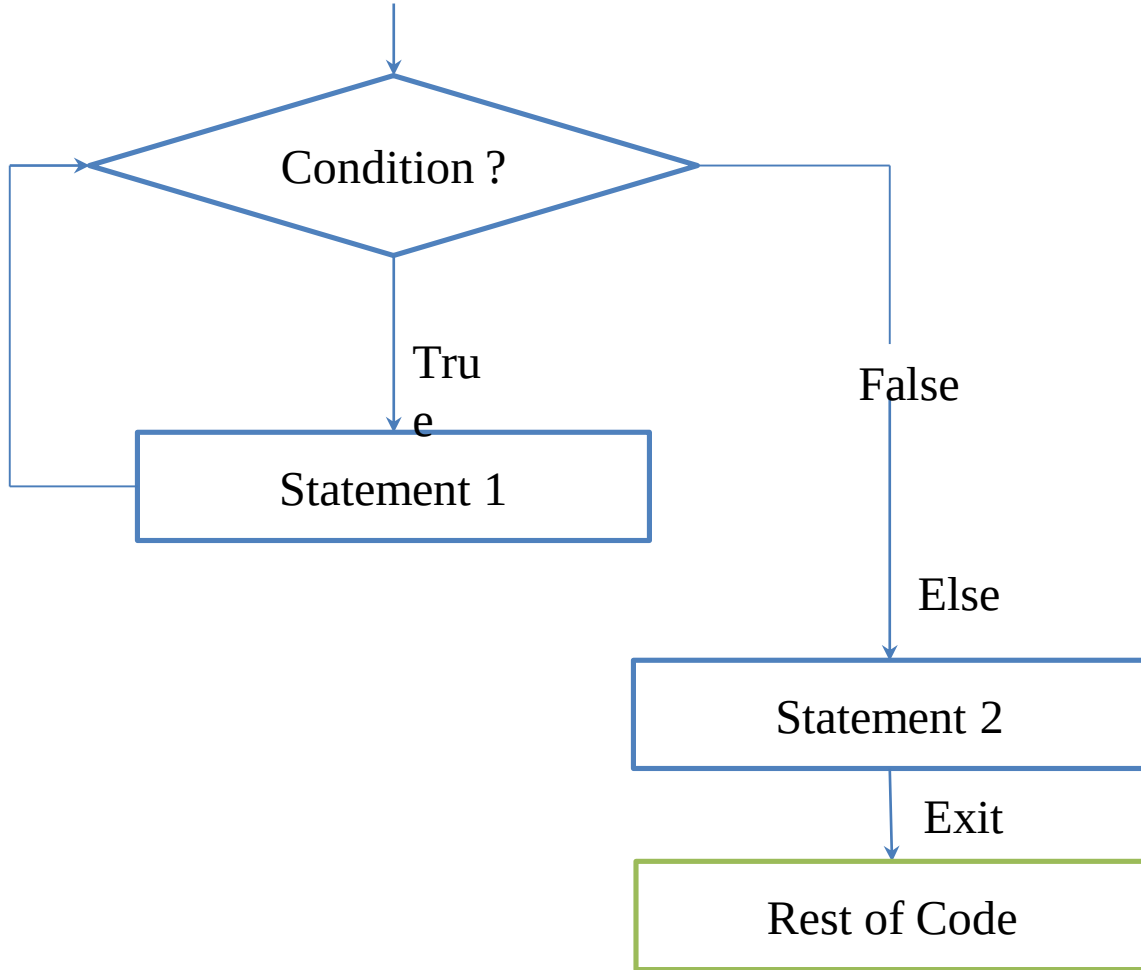


while Loop with else

This repeatedly tests the condition and, if it is True, executes the Statement 1; if the condition is False (which may be the first time it is tested) the Statement 2 of the else clause, is executed and the loop terminates. The else suite will be always executed irrespective of the statements in the loop are executed or not.

Syntax:

```
while (condition):  
    Statement 1  
else:  
    Statement 2  
Rest of the Code
```



Infinite while Loop

Syntax:

```
while (True):  
    Statement  
Rest of the Code
```

```
while (True):  
    Statement  
    if(condition):  
        break  
Rest of the Code
```

Nested While Loop

```
while(condition):
```

```
    Statements
```

```
        while(condition):
```

```
            Statements
```

```
        Statements
```

```
Rest of Code
```

range() Function

range() function is used to generate a sequence of integers starting from 0 by default, and increments by 1 by default, till j-1.

Syntax:-

range(start, stop, stepsize)

Start – Starting position. If we do not mention start by default it's 0

*Stop – Ending position. The range of integers stops one element prior to stop. If stop is j then it will stop at exact j-1

Stepsize – Increment by stepsize. If we do not mention start by default it's 1

range() Function

Syntax:- range(j) 0, 1, 2, 3, 4,....., j-1

Ex:- range(10) 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Syntax:- range(i, j) i, i+1, i+2, i+3,....., j-1

Ex:- range(1, 10) 1, 2, 3, 4, 5, 6, 7, 8, 9

Syntax: - range(i, j, k) i, i+k, i+2k, i+3k, i+4k,....., j-1

range(1, 10, 2) 1, 3, 5, 7, 9

range(-1, -10, -2) -1 -3 -5 -7 -9

range(10, 0, -1) 10 9 8 7 6 5 4 3 2 1

Rules:-

- All argument must be integers, whether its positive or negative
- You can not pass a string or float number or any other type in a start, stop and stepsize.
- The stepsize value should not be zero.

for Loop

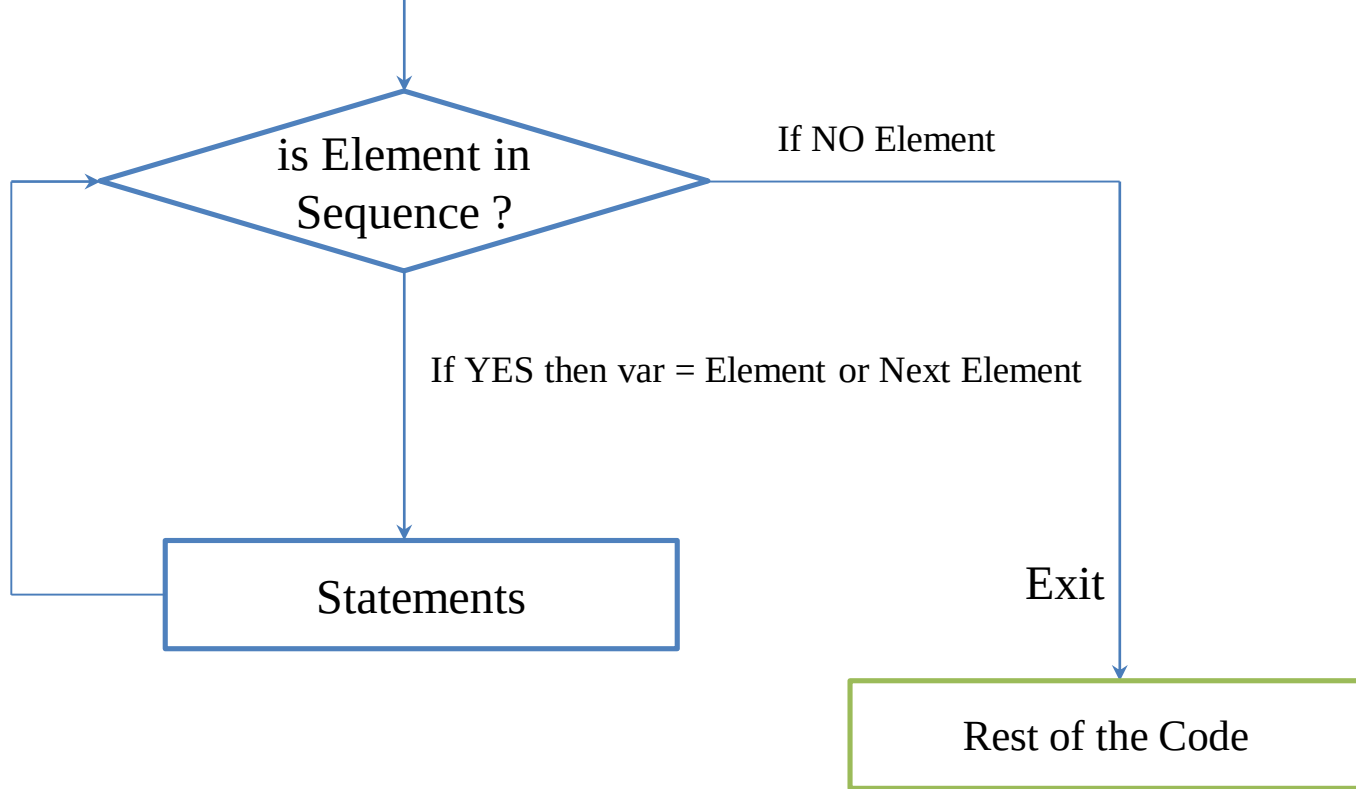
The for loop is useful to iterate over the elements of sequence such as string, list, tuple etc.

Syntax:

```
for var in sequence:
```

```
    Statements
```

```
Rest of the Code
```



for Loop with Range

```
a = range(5)
```

```
for i in a :
```

```
    print(i)
```

```
for i in range(5) :
```

```
    print(i)
```

for Loop with else

The for loop is useful to iterate over the elements of sequence such as string, list, tuple etc. The else suite will be always executed irrespective of the statements in the loop are executed or not.

Syntax:

for var in sequence:

Statements

else:

Statements

Rest of the Code

Nested for loop

For loop inside another for loop is known as nested for loop.

```
for i in range(n):
```

```
    for j in range(y):
```

```
        Statements
```

```
    Statements
```

```
Rest of the Code
```

Break Statement

Break statement is used to jump out of loop to process the next statement in the program.

```
while condition:
```

```
    if(condition):
```

```
        break
```

```
Rest of Code
```


Continue Statement

Continue statement is used in a loop to go back to the beginning of the loop.

while condition:

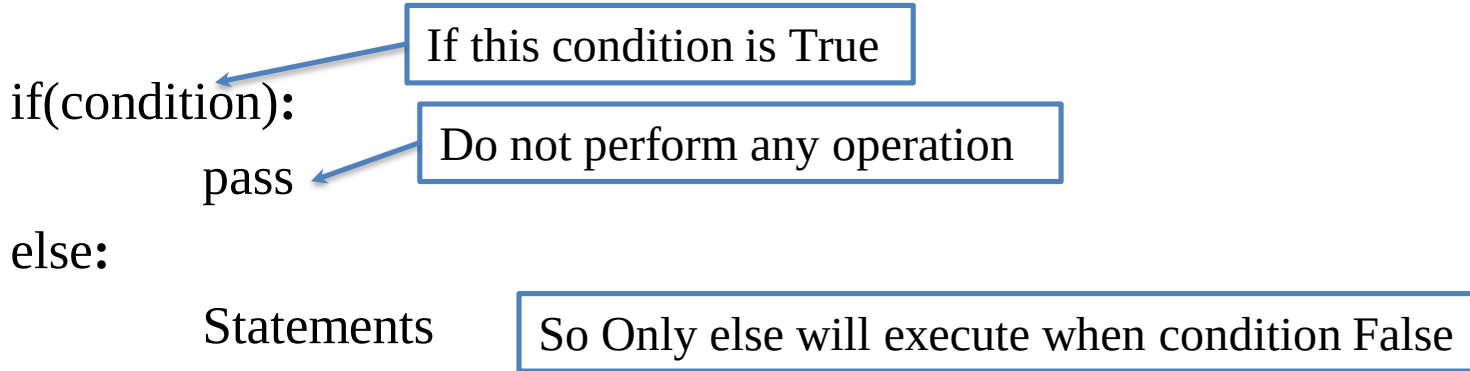
 if(condition):

 continue

Rest of Code

Pass Statement

Pass Statement is used to do nothing. It can be used inside a loop or if statement to represent no operation. Pass is useful when we need statement syntactically correct but we do not want to do any operation.



Pass Statement

while condition:

 if(condition):

 pass

 Statements

Rest of Code

List of Tuples in Python

```
students = [  
    ("Alice", 23, "Math"),  
    ("Bob", 21, "Physics"),  
    ("Charlie", 22, "Computer Science")  
]
```

```
# Print all names
```

```
for name, age, subject in students:
```

```
    print(f"{name} studies {subject} and is {age} years old.")
```

Tuple of list

```
sales = (  
    [200, 220, 250], # Week 1 sales  
    [180, 210, 230], # Week 2 sales  
    [300, 280, 320]  # Week 3 sales  
)  
  
# Find total sales per week  
totals = [sum(week) for week in sales]  
print("Weekly Totals:", totals)
```

Nested Dictionary

```
company = {  
    "HR": {"employees": ["Alice", "Bob"], "manager": "Sarah"},  
    "IT": {"employees": ["Charlie", "David"], "manager": "John"},  
    "Finance": {"employees": ["Eve"], "manager": "Emma"}  
}
```

```
print(company["IT"]["employees"]) # ['Charlie', 'David']
```

What are Comprehensions?

Comprehensions are a concise way to create sequences (lists, sets, dicts) or apply transformations using a single line.

There are mainly 4 types:

1. List Comprehension
2. Set Comprehension
3. Dictionary Comprehension
4. Generator Comprehension

List Comprehension

Normal way

```
squares = []
```

```
for i in range(5):
```

```
    squares.append(i * i)
```

List comprehension

```
squares = [i * i for i in range(5)]
```

```
print(squares) # [0, 1, 4, 9, 16]
```

With Condition:

```
evens = [i*i for i in range(10) if i % 2  
== 0]
```

```
print(evens) # [0, 2, 4, 6, 8]
```


Set Comprehension

```
nums = [1, 2, 2, 3, 4, 4, 5]  
unique_squares = {x * x for x in nums}  
print(unique_squares) # {1, 4, 9, 16, 25}
```

Dictionary Comprehension

Squares dictionary

```
squares = {x: x * x for x in range(5)}
```

```
print(squares) # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

```
even_squares = {x: x * x for x in range(10) if x % 2 == 0}
```

```
print(even_squares) # {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

Generator Comprehension

```
gen = (x * x for x in range(5))
```

```
print(next(gen)) # 0
```

```
print(next(gen)) # 1
```

```
print(list(gen)) # [4, 9, 16]
```

What is a Package?

- A **package** in Python is a **collection of modules** (Python files) organized in a **directory**.
- It helps you **organize and reuse code**.
- A package usually contains an `__init__.py` file (can be empty, but it marks the folder as a package)..

Ex:

```
import numpy as np
```

```
arr = np.array([1, 2, 3])  
print(arr * 2)  # [2 4 6]
```